



Lớp Shape (abstract) — ABC + abstract method

Cài đặt `class Shape` là **abstract class** — không cho khởi tạo trực tiếp; method `area()` là abstract. `Circle` kế thừa `Shape` với bán kính `r` (int); `Rectangle` kế thừa `Shape` với `w, h` (int). Mỗi subclass cài `area()` trả về `double`.

Python: `class Shape(ABC):, @abstractmethod def area(self): ....` Cố gọi `Shape()` trực tiếp → `TypeError`. Dùng `math.pi` cho π .

C++: dùng **pure virtual**: `virtual double area() const = 0;` trong `Shape` (đã có sẵn skeleton). Subclass override `double area() const override`. Dùng `M_PI` (đã được runner `#define` sẵn). Driver tạo `Shape*` trở về object con — cấm khởi tạo `Shape` trực tiếp do có pure virtual.

□ **Bạn chỉ nộp khai báo lớp**

File `.py` bạn nộp chỉ chứa **Shape + Circle + Rectangle** — **không** viết `main`, `if __name__`, hay driver đọc stdin. Grader **lấy tên file bạn nộp** làm tên module, **import lớp** của bạn, **chạy driver** để ném stdin vào và so sánh stdout.

□ **Đặt tên file** là tên identifier hợp lệ (vd. `Point.py`, `TuLanh.py`, `Shape.py`). Nếu đặt bậy (`my-bai.py`, `1.py`) grader fallback `student_solution`.

Luồng chấm bài

`Shape.py` (bạn nộp) → `from Shape import *` → driver grader → stdin → stdout → so khớp = pass

□ Khung lớp bạn nộp lên

PythonC++

```
from abc import ABC, abstractmethod
```

```
import math
```

```
class Shape(ABC):
```

```
    @abstractmethod
```

```
def area(self) -> float:
```

```
    ...
```

```
class Circle(Shape):
```

```
    def __init__(self, r: int):
```

```
        self.r = r
```

```
    # TODO: area() → math.pi * r * r
```

```
class Rectangle(Shape):
```

```
    def __init__(self, w: int, h: int):
```

```
        self.w, self.h = w, h
```

```
    # TODO: area() → w * h
```

```
class Shape {
```

```
public:
```

```
    virtual ~Shape() {}
```

```
    virtual double area() const = 0;  // pure virtual
```

```
};
```

```
class Circle : public Shape {
```

```
    int r_;
```

```

public:

    Circle(int r) : r_(r) {}

    // TODO: double area() const override { return M_PI * r_ * r_; }

};

class Rectangle : public Shape {

    int w_, h_;

public:

    Rectangle(int w, int h) : w_(w), h_(h) {}

    // TODO: double area() const override { return w_ * h_; }

};

```

▢ Grader sẽ làm gì với lớp của bạn

1. Đọc N — số hình
2. Mỗi dòng: `circle r (r: int)` hoặc `rect w h (w, h: int)`
3. Tính `area()` mỗi hình, in **2 chữ số thập phân** — Py:


```
print(f'{s.area():.2f}') / C++: std::cout << std::fixed <<
std::setprecision(2) << s->area()
```
4. Hằng π : Py `math.pi`, C++ `M_PI`

► Xem code driver grader tự chạy

Grader chạy đoạn code sau cùng file bạn nộp. Dùng nút ở khung khai báo để xem driver cho từng ngôn ngữ:

```

from Shape import * # file SV nộp

import math

n = int(input())

```

```

for _ in range(n):

    parts = input().split()

    if parts[0] == 'circle':

        s = Circle(int(parts[1]))

    else:

        s = Rectangle(int(parts[1]), int(parts[2]))

    print(f'{s.area():.2f}')

```

```

#include <iostream>    // grader tự prepend khi compile

```

```

// ... + class bạn viết ...

```

```

int main() {

    int n; std::cin >> n;

    std::cout << std::fixed << std::setprecision(2);

    for (int i = 0; i < n; ++i) {

        std::string kind; std::cin >> kind;

        Shape* s = nullptr;

        if (kind == "circle") { int r; std::cin >> r; s = new Circle(r); }

        else { int w, h; std::cin >> w >> h; s = new Rectangle(w, h); }

        std::cout << s->area() << std::endl;

        delete s;
    }
}

```

```
}  
  
    return 0;  
  
}
```

□ Testcase mẫu — để đối chiếu format

Grader chạy *code SV + driver* rồi so sánh stdout từng dòng (strip whitespace cuối, bỏ dòng trống thừa ở cuối). Các testcase ẩn có cùng định dạng.

Ví dụ 1

stdin

```
2  
  
circle 5  
  
rect 3 4
```

expected stdout

```
78.54  
  
12.00
```

Ví dụ 2

stdin

```
3  
  
rect 10 10  
  
circle 1  
  
rect 7 3
```

expected stdout

```
100.00
```

3.14

21.00

Ngôn ngữ hỗ trợ: bài này nhận `.py`/`.cpp`. Grader tự ghép driver + class bạn nộp rồi chạy. Java sẽ mở khi có driver tương ứng.

□

Đây là bài **thực hành** — bạn **nộp lại bao nhiêu lần cũng được**. Mỗi lần nộp, hệ thống giữ điểm cao nhất. Thử, đọc error, fix, thử tiếp.